



Université Mohammed V de Rabat

Ecole Supérieure de Technologie de Salé

Architecture des Web services REST

Pr R.ALAOUI

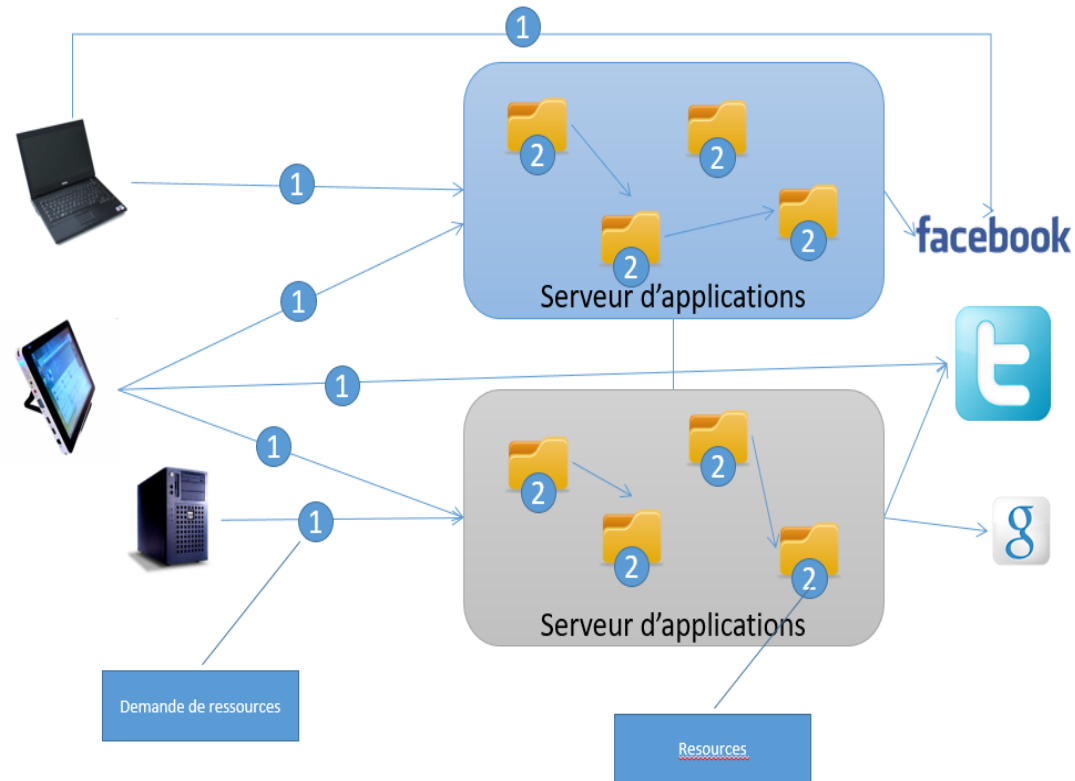
Introduction aux Web Services

Utilisation du web aujourd'hui

Les **web services** permettent l'interaction entre différentes applications via Internet.

Les utilisateurs (ordinateurs, tablettes, serveurs) envoient des **demandes de ressources** (1) à des **serveurs d'applications**.

Les **serveurs d'applications** gèrent les requêtes et fournissent des **ressources** (2), qui peuvent être des données ou des services, comme ceux de **Facebook**, **Twitter**, ou **Google**.



Les **web services** permettent l'interaction entre différentes applications via HTTP

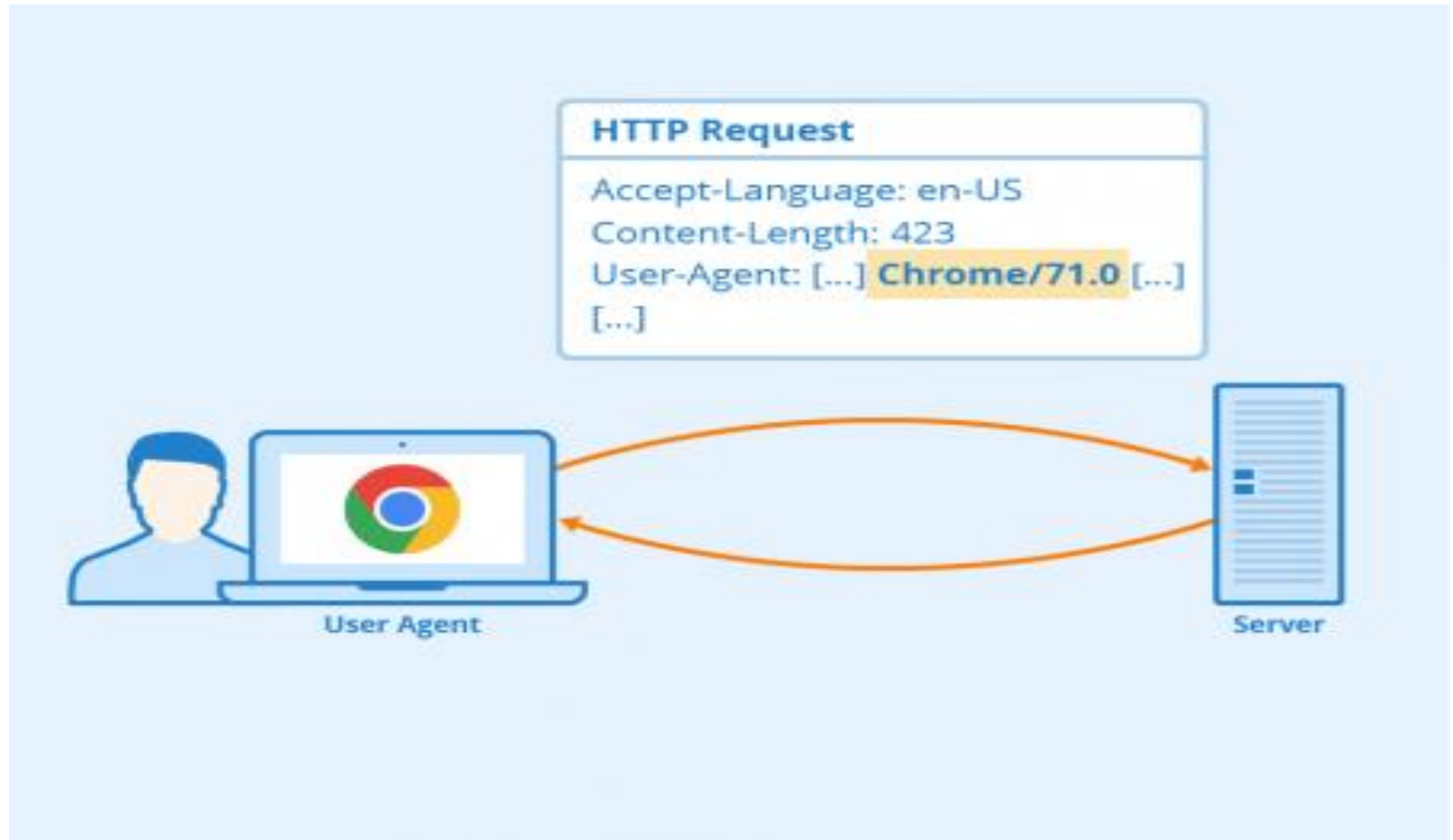
- *HyperTextTransfer Protocol*
 - à l'origine, protocole permettant de publier et de retrouver des pages
- Version : HTTP/1.1 largement utilisée aujourd'hui sur le Web
- Construit au-dessus de TCP, port 80 par défaut (ou 443 en SSL)
- Protocole requête/réponse **sans état**

Terminologie

Le serveur est appelé *origin server*

Le client est appelé *user-agent*

Le serveur offre des *resources* qui peuvent être accédées grâce aux URIs



Format des URLs

Une **URL** (Uniform Resource Locator) est souvent définie comme une **chaîne de caractères pointant vers une adresse**.

Elle permet de retrouver la **représentation d'un emplacement physique** en **décrivant** son **emplacement réseau** ou son principal mécanisme d'accès.

<http://serveur:port/chemin/vers/la/ressource?p1=v1&p2=v2>

The diagram shows a general URL structure with three parts separated by double-headed arrows. The first part is 'protocole+serveur' (protocol+server), the second is 'chemin' (path), and the third is 'requête' (query).

Chaque ressource unique (image, page HTML, document CSS, etc.) est toujours représentée par une URL.

<https://www.geeksforgeeks.org/php-function>

The diagram shows a specific URL structure with three parts separated by double-headed arrows. The first part is 'Protocol', the second is 'hostname', and the third is 'filename'.

Syntaxe d'une URL

Chaque URL HTTP respecte la syntaxe de son URI générique. Par conséquent, la syntaxe de l'URL et celle de l'URI sont comparables. Elle est présentée ci-dessous :

- **Schéma** : **protocole** qu'un navigateur doit utiliser (HTTP et HTTPS)
- **Autorité** : Le **nom de domaine** et le port, deux sous-composantes de l'autorité, sont séparés par deux points.
- **Chemin** : Le chemin sur le serveur web indique l'itinéraire complet vers la ressource.
- **Chaîne de requête** : La chaîne contenant la paire nom-valeur est appelée chaîne de requête.
Lorsqu'elle apparaît dans une URL, les informations sont fournies en suivant le composant de chemin. « ?key1=value1&key2=value2 » en est un exemple.
- **Fragment (Ancre)**: élément optionnel placé avant le symbole dièse (#). Il contient un identifiant de fragment qui pointe vers une ressource de sauvegarde.



URI (Uniform Resource Identifier)

C'est l'identifiant d'une ressource.

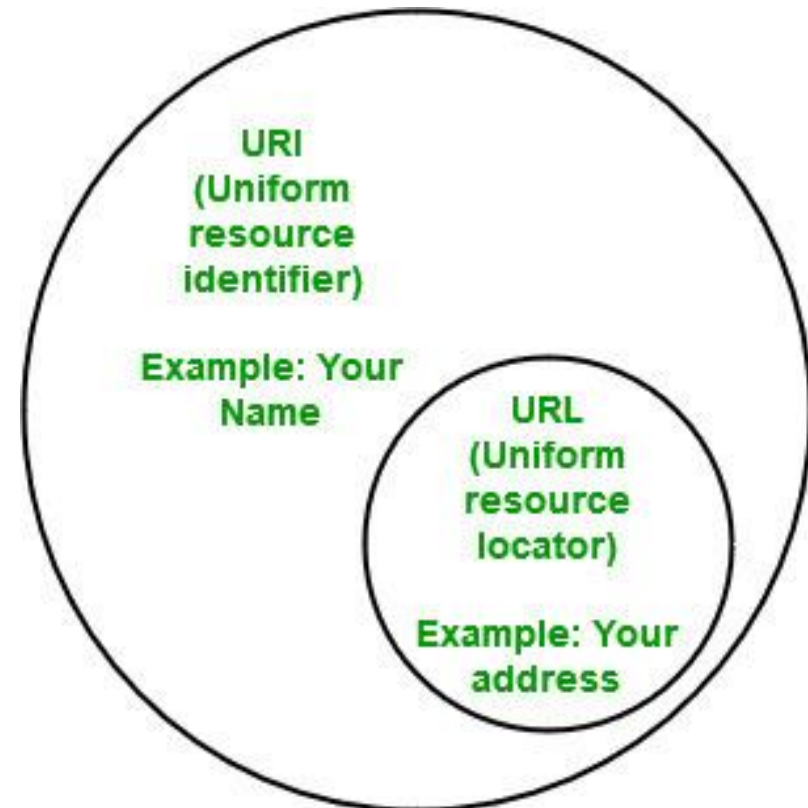
Un **URI** sert à **nommer** / **identifier** quelque chose sur le réseau, même si on ne dit pas forcément *comment* y accéder.

☞ **Exemple d'URI :**

urn:isbn:9780132350884 (identifie un livre par ISBN, **sans** donner un emplacement web)

Différence essentielle

- **URI = identifie** une ressource (concept plus large)
- **URL = localise** une ressource **et** indique comment y accéder
-
- ✓ Donc : **toute URL est un URI**, mais **tous les URI ne sont pas des URL**.



Syntaxe d'une URI

Syntaxe d'une URI

URI = schéma://autorité/chemin?requête#fragment

1. Schéma (scheme)

Indique le protocole utilisé.

Ex. : http, https, ftp

→ Exemple : https://

2. Autorité (authority)

Identifie le serveur. Elle contient généralement :

- **Hôte (host)** : nom de domaine ou IP (ex. api.exemple.com)

- **Port (optionnel)** : (ex. :8080)

- *(optionnel et rare en REST)* **user:password@**

→ ☐ Exemple : api.exemple.com:8080

3. Chemin (path)

Le “chemin” vers la ressource REST (souvent au pluriel).

→ ☐ Exemple : /users/15/orders

4. Requête (query) – optionnel (?)

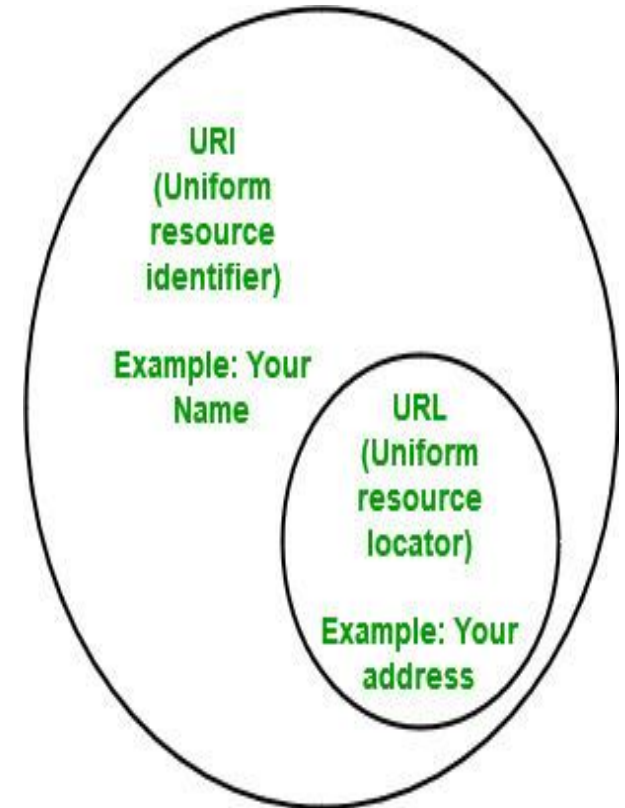
Paramètres supplémentaires (filtrage, tri, pagination...).

→ ☐ Exemple : ?page=2&size=10&sort=name

5. Fragment – optionnel (#)

Pointe vers une partie d'un document côté client (souvent HTML).

→ ☐ En REST API, **peu utilisé**. Exemple : #section2



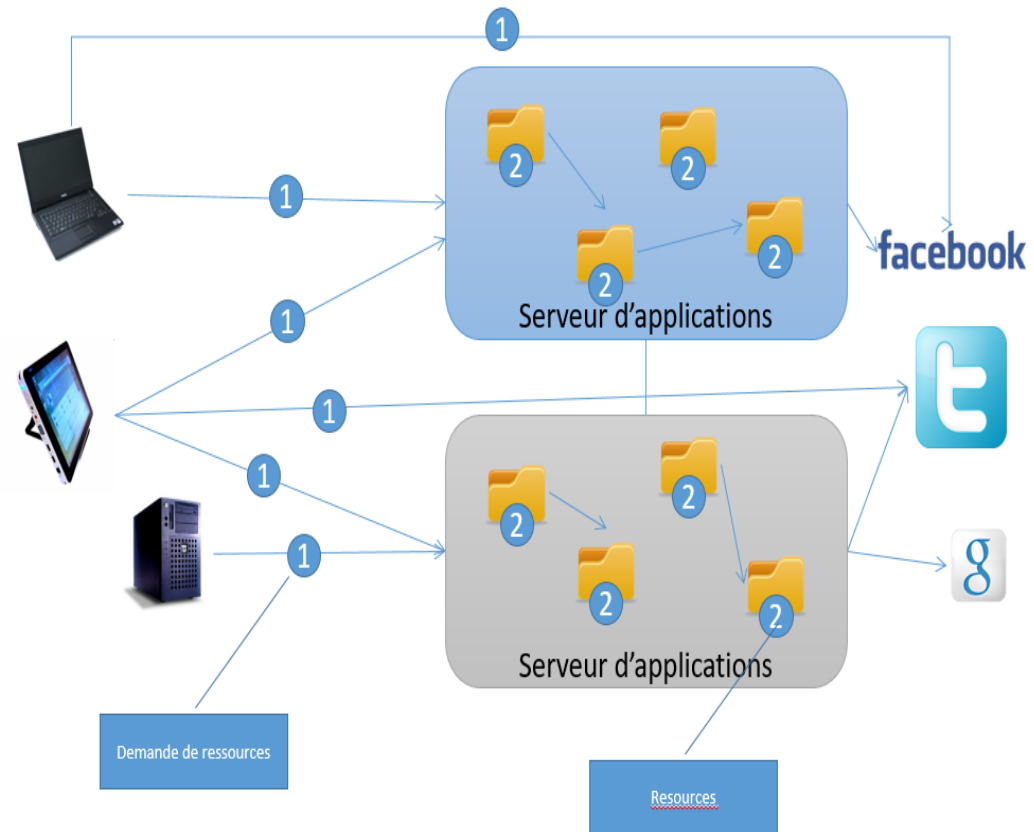
Web Service REST

Les **web services REST** (Representational State Transfer) sont particulièrement adaptés à ce modèle. REST repose sur les principes suivants :

1.Architecture client-serveur : Les utilisateurs (clients) communiquent avec des serveurs pour accéder aux ressources.

1.Stateless : Chaque requête est indépendante, ce qui simplifie la scalabilité.

2.Utilisation de méthodes HTTP : Les opérations comme GET (lecture), POST (création), PUT (mise à jour), et DELETE (suppression) permettent d'interagir avec les ressources.



Définition

- ❑ Acronyme de **RE**presentational **S**tate **T**ransfert défini dans la thèse de Roy Fielding en 2000.
- ❑ REST n'est pas un protocole ou un format, contrairement à SOAP, HTTP, mais un **style d'architecture** inspiré de l'architecture du web **fortement basé** sur le **protocole HTTP**
- ❑ Il ne dépend pas uniquement du web et peut utiliser d'autres protocoles que HTTP

Web Service REST

Ce qu'il est :

- Un système d'architecture
- Une approche pour construire une application

Ce qu'il n'est pas

- Un protocole
- Un format
- Un standard

REST ➔ utilisation

- Utiliser dans le développement des applications orientés ressources (ROA) ou orientées données (DOA)
- Les applications respectant l'architecture REST sont dites RESTful

REST → Fournisseurs

facebook



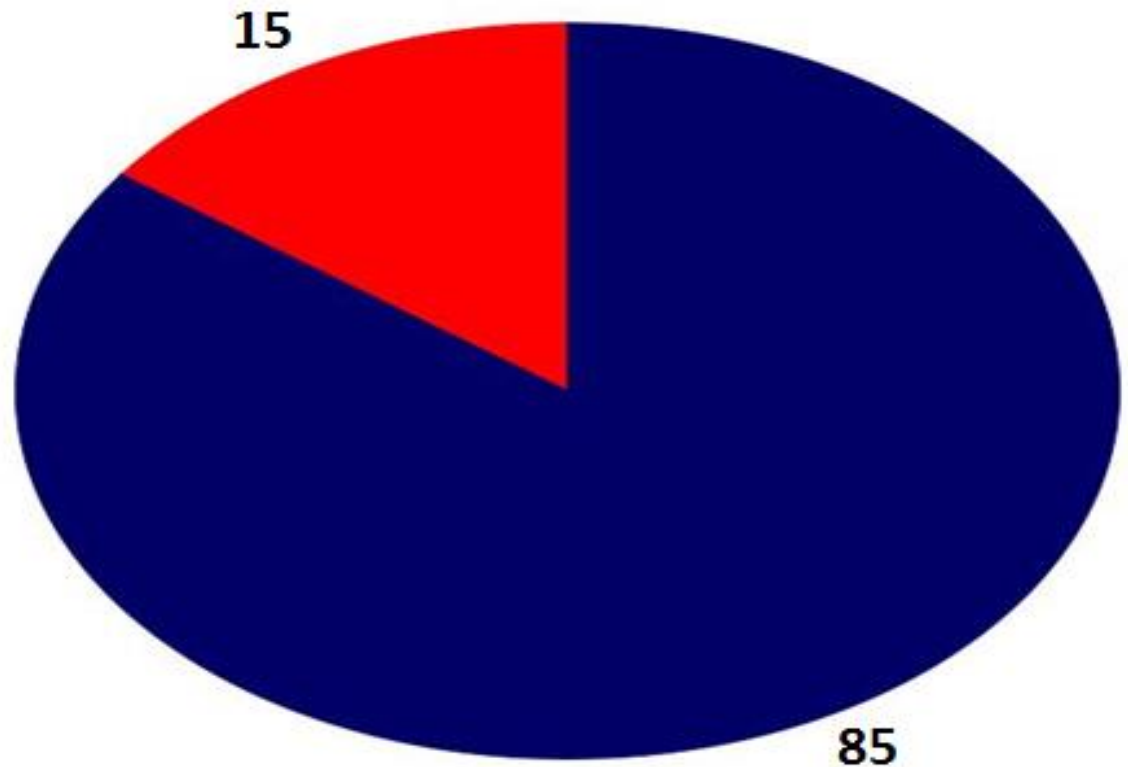
Google



REST → Statistics

Statistique d'utilisation des services web REST et SOAP chez AMAZON

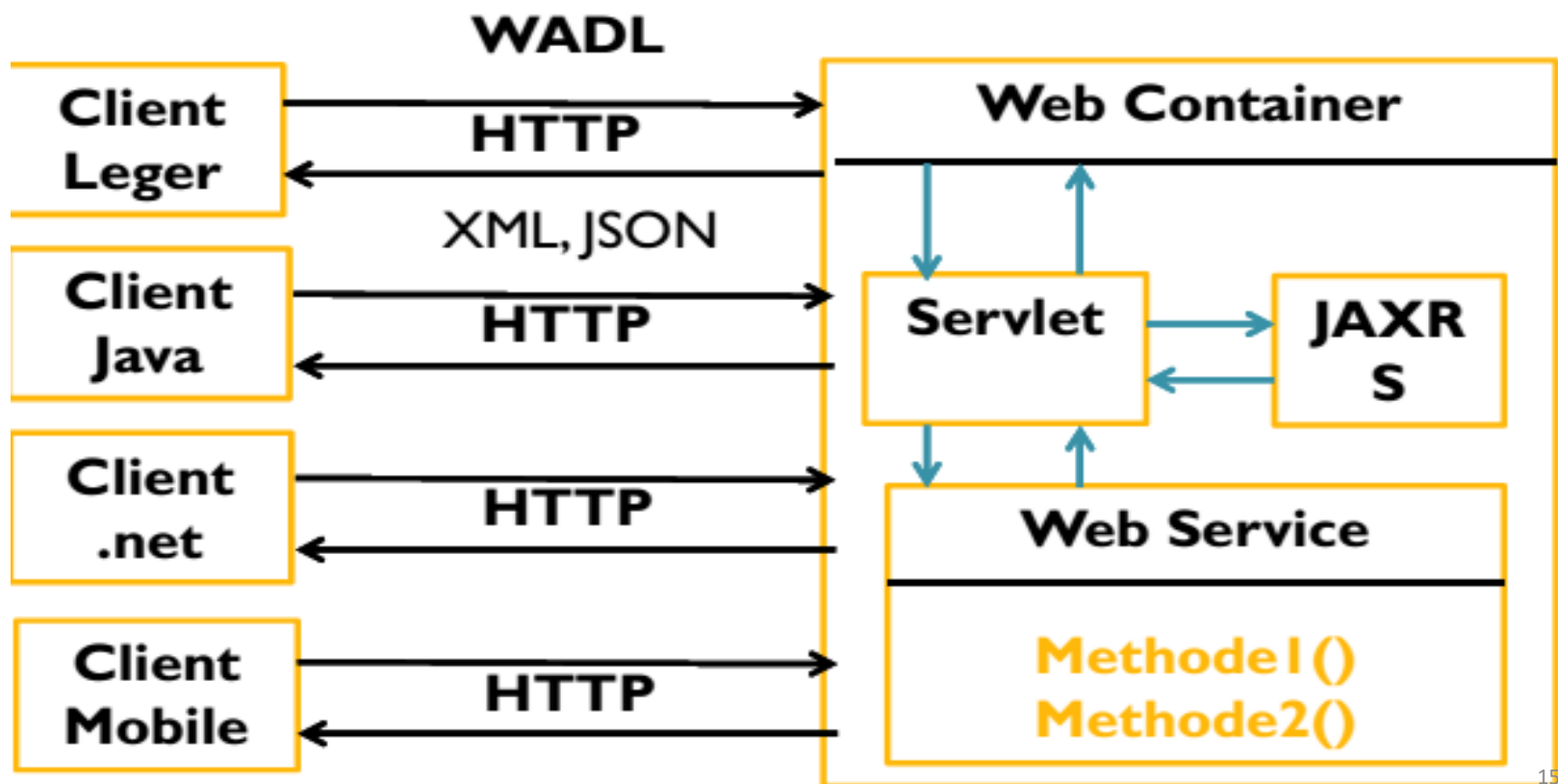
■ REST
■ SOAP



REST: Principe

Une API REST suit des conventions standards pour garantir que les clients comprennent toujours comment interagir avec le serveur

Description du Service Web permettant de générer la partie cliente



URI (**U**niform **R**esource **I**dentifier)

C' est une **adresse unique** qui **identifie une ressource** sur un **serveur**.

URI c'est un **élément clé** de l'**architecture REST**, car il permet de **localiser des ressources spécifiques** à travers le web.

Structure d'un URI

Un URI est composé de **plusieurs parties** :

<https://example.com/api/users/123>

Protocole : https Définit le protocole de communication (HTTP ou HTTPS pour REST).

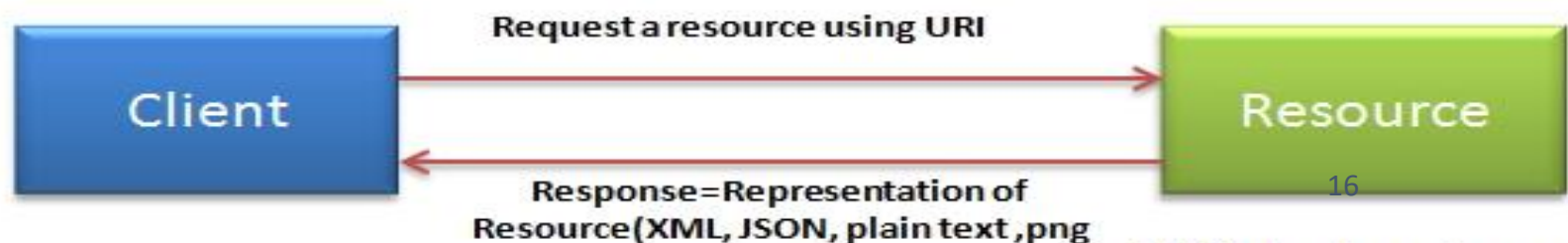
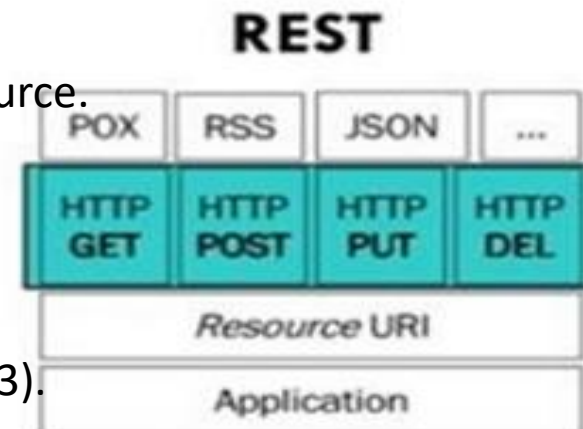
Domaine : **example.com** Identifie le serveur qui héberge la ressource.

Chemin : **/api/users/123** Spécifie l'**emplacement exact** de la **ressource sur le serveur**.

/api : Indique qu'il s'agit d'un point d'accès à une API.

/users : Définit une collection de ressources (utilisateurs).

/123 : Identifie une ressource spécifique (l'utilisateur avec l'ID 123).



Règle N°1 :

l'URI comme identifiant des ressources

- REST se base sur les **URI (Uniform Resource Identifier)** afin d'identifier une ressource.
- Ainsi une application se doit de construire ses URI (et donc ses URL) de manière précise, en tenant compte des contraintes REST.
- Il est nécessaire de prendre en compte la hiérarchie des ressources et la sémantique des URL pour les éditer :

REST ➡ Caractéristiques

- Les services REST sont sans états (Stateless)
 - Chaque requête envoyée au serveur doit contenir toutes les informations relatives à son état et est traitée indépendamment de toutes autres requêtes
 - Minimisation des ressources systèmes (pas de gestion de session, ni d'état)
- Interface uniforme basée sur les méthodes HTTP (GET, POST, PUT, DELETE)
- Les architectures RESTful sont construites à partir de ressources uniquement identifiées par des URI(s)

Requêtes REST

- **Ressources**

- Identifiée par une URI (<http://est.um5.ac.ma/est/IAM>)

- **Méthodes (verbes)** permettant de manipuler les ressources (identifiants)

- Méthodes HTTP : **GET, POST, PUT, DELETE**

- **Représentation** : Vue sur l'état de la ressource

- Format d'échanges entre le client et le serveur (XML, JSON, text/plain,...)

Ressources

❑ Une ressource est un objet identifiable sur le système

➔ Livre, Catégorie, Client, Prêt

Une ressource n'est pas forcément un objet matérialisé (Prêt, Consultation, Facture...)

❑ Une ressource est identifiée par une URI : Une URI identifie uniquement une ressource sur le système ➔ une ressource peut avoir plusieurs identifiants

Une requête GET sur **/users/123**

peut retourner cette ressource au format JSON :

Json

```
{  
  "id": 123,  
  "name": "Alice Dupont",  
  "email": "alice.dupont@example.com",  
  "role": "Admin "  
}
```

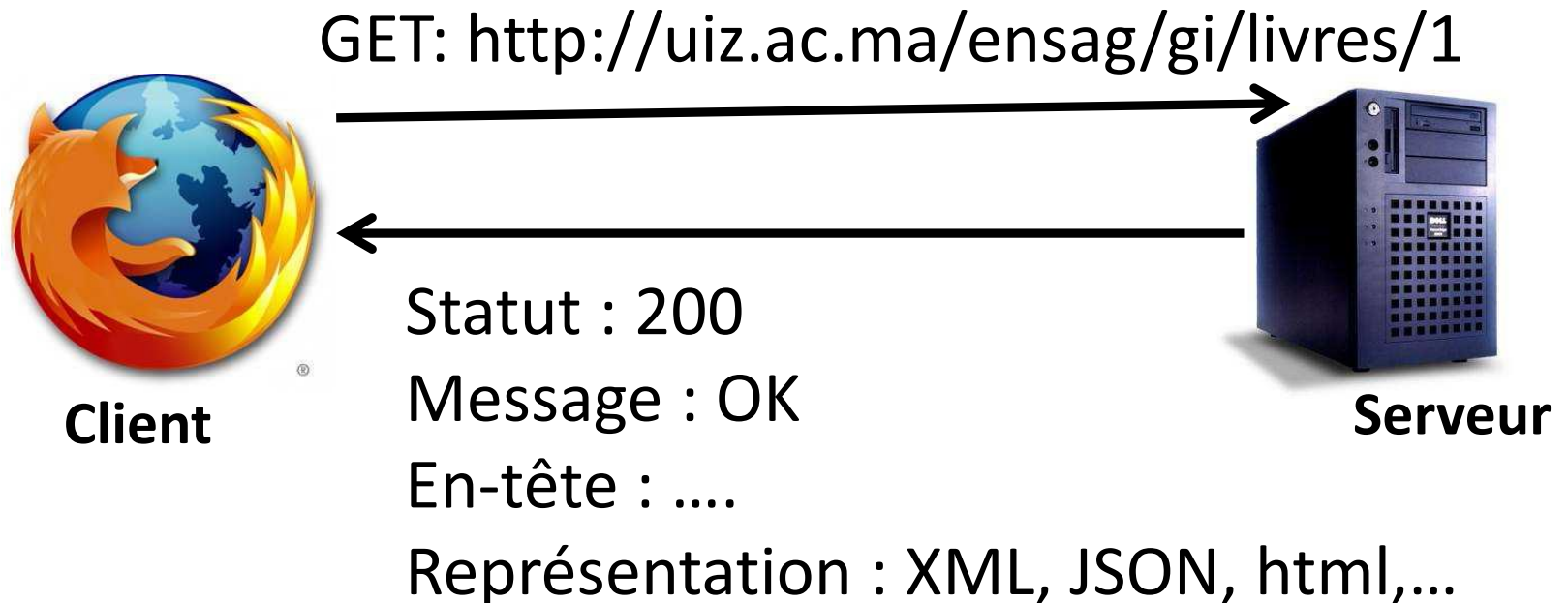
Methodes (Verbes)

- Une ressource peut subir quatre opérations de bases CRUD correspondant aux quatre principaux types de requêtes HTTP (GET, PUT, POST, DELETE)
- REST s'appuie sur le protocole HTTP pour effectuer ces opérations sur les objets
 - **CREATE → POST**
 - **RETRIEVE → GET**
 - **UPDATE → PUT**
 - **DELETE → DELETE**

Rappel: CRUD (*Create, Read, Update, Delete*) désigne les **quatre opérations de base** pour la **persistance** des données, en particulier le **stockage d'informations** en **base de données**.²⁴

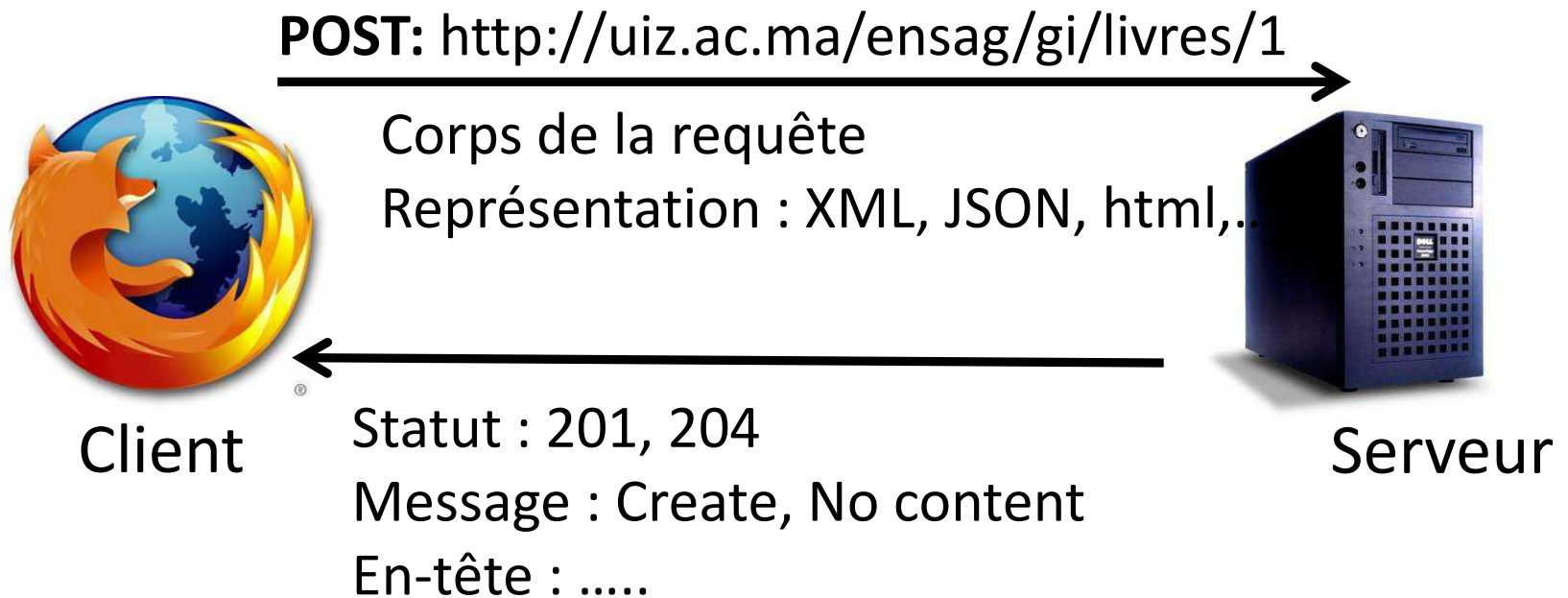
Méthode GET

- La méthode GET renvoie une représentation de la ressource tel qu'elle est sur le système



Méthode POST

- La méthode POST crée une nouvelle ressource sur le système



Méthode DELETE

- Supprime la ressource identifiée par l'URI sur le serveur



Méthode PUT

- Mise à jour de la ressource sur le système

Identifiant de la
ressource sur le
serveur

PUT: <http://uiz.ac.ma/ensag/gi/livres/1>



Client

En-tête :

Corps de la requête : XML, JSON,...



Serveur

Statut : 200

Message : OK

En-tête :

JAX-RS : Java API for RESTful Web Services 🌐

JAX-RS (Java API for RESTful Web Services) est une **spécification** de Java utilisée pour développer des web services RESTful.

JAX-RS fait partie de la plateforme **Java EE** (ou Jakarta EE depuis sa transition) et permet de créer des applications REST en utilisant des annotations simples et des concepts orientés objet.

Objectif de JAX-RS 🚀

Fournir un **moyen facile et standardisé** pour **développer des API RESTful** avec Java, en **simplifiant l'échange** de données entre un **client** (navigateur, application mobile, etc.) et un **serveur**.

Fonctionnement de JAX-RS ✂

Annotations :

JAX-RS repose sur des annotations qui permettent de définir facilement des points d'accès REST.

Ces annotations simplifient la création et la gestion des routes.

1. **@Path** : Définit le chemin d'accès à une ressource.
2. **@GET, @POST, @PUT, @DELETE** : Mappent les verbes HTTP à des méthodes Java.
3. **@Produces** et **@Consumes** : Indiquent les types de données que l'API peut envoyer ou recevoir (par exemple, JSON, XML, etc.).

JAX-RS : la spécification

- Différentes implémentations de la spécification JAX-RS sont disponibles
 - **JERSEY** : implémentation de référence fournie par Oracle Site projet : <http://jersey.java.net>
 - **CXF** : fournie par Apache, la fusion entre XFire et Celtix Site projet : cxf.apache.org
 - **RESTEasy** : fournie par JBoss Site projet : www.jboss.org/resteasy
 - **RESTlet** : un des premiers framework implémentant REST pour Java : Site projet : www.restlet.org

Représentation

Une représentation désigne les données échangées entre le client et le serveur pour une ressource

Peut être sous différent format :

- **JSON**
- **XML**
- **XHTML**
- **CSV**
- **Text/plain**
-

JSON

JSON « **J**ava**S**cript **O**bject **N**otation » est un format d'échange de données, facile à lire par un humain et interpréter par une machine.

Basé sur JavaScript, il est complètement indépendant des langages de programmation mais utilise des conventions qui sont communes à toutes les langages de programmation (C, C++, Perl, Python, Java, C#, VB, JavaScript,...)

Deux structures :

- **Une collection de clefs/valeurs → Object**
- **Une collection ordonnée d'objets → Array**

Exemple XML/JSON

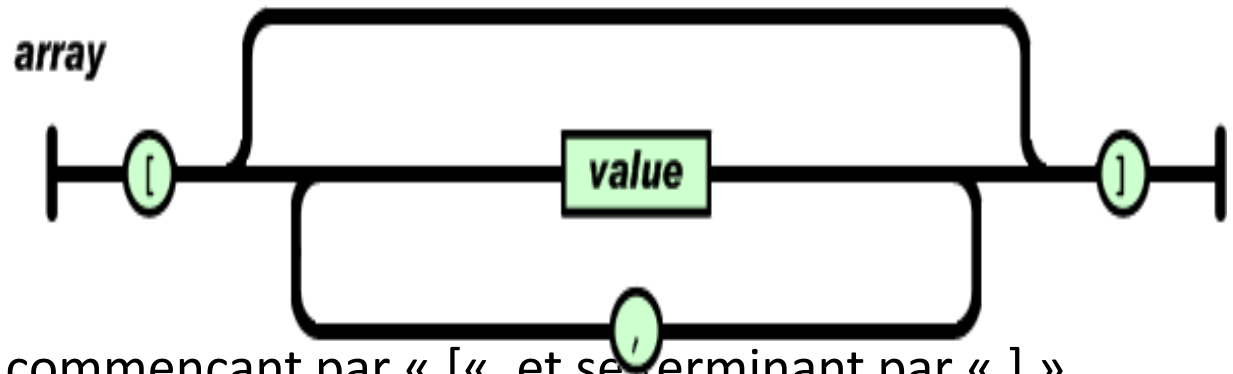
XML

```
<?xml version="1.0" encoding="UTF-8"?>
  <comptes>
    <compte type="courant">
      <code>1</code>
      <solde>4300</solde>
      <dateCreation>2012-11-11</dateCreation>
    </compte>
    <compte type="epargne">
      <code>2</code>
      <solde>96000</solde>
      <dateCreation>2012-12-11</dateCreation>
    </compte>
  </comptes>
```

JSON

```
[
  {type:"courant" , code:1 , solde:4300.50 ,dateCreation:"2012-11-11"},
  {type:"epargne" , code:1 , solde:4300.00 ,dateCreation:"2012-11-11"}
]
```

JSON



ARRAY

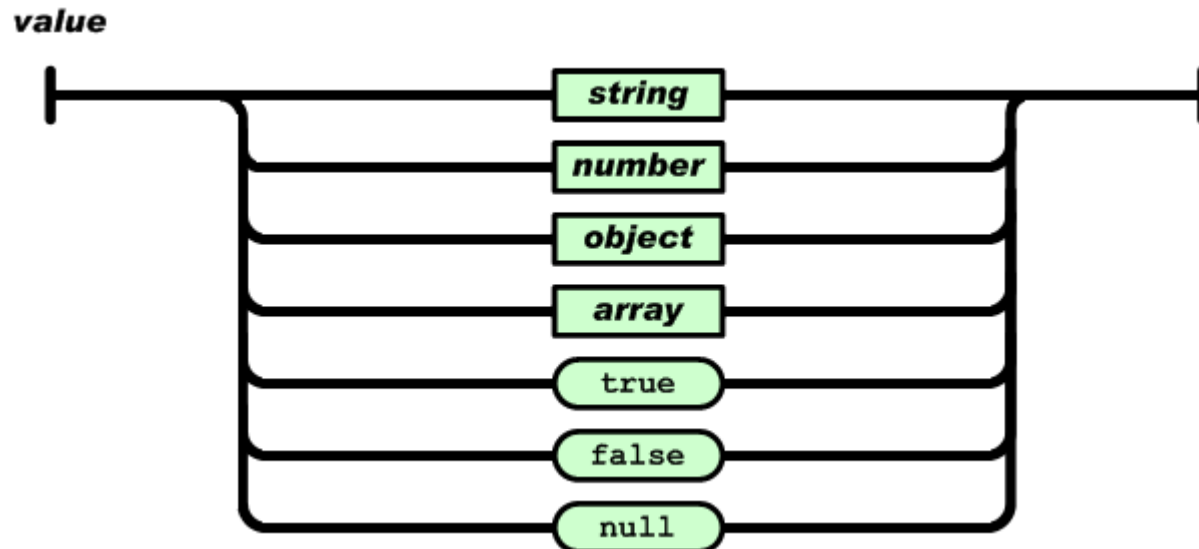
Liste ordonnée d'objets commençant par « [» et se terminant par «] », les objets sont séparés l'un de l'autre par « , ».

```
[
  { "id": 51,
    "nom": "Mathematiques 1", "resume":
    "Resume of math ",
    "isbn": "123654", "quantite": 42,
    "photo": ""
  },
  { "id": 102,
    "nom": "Mathematiques 1",
    "resume": "Resume of math ",
    "isbn": "12365444455", "quantite": 42,
    "photo": ""
  }
]
```


JSON

Value

Un objet peut être soit un string entre « "" » ou un nombre (entier, décimal) ou un boolean (true, false) ou null ou un objet.



JAX-RS : Développement

- Basé sur POJO (Plain Old Java Object) en utilisant des annotations spécifiques JAX-RS
- Pas de modifications dans les fichiers de configuration
- Le service est déployé dans une application web
- Pas de possibilité de développer le service à partir d'un WADL contrairement à SOAP
- Approche Bottom/Up
 - Développer et annoter les classes
 - Le WADL est automatiquement généré par l'API

Annotations JAX-RS :

@Path :

Spécifie l'URI d'une ressource.

@GET, @POST, @PUT, @DELETE :

Mappent les méthodes HTTP à des méthodes Java.

@Produces :

Indique le type de réponse (JSON, XML, etc.).

@Consumes :

Indique le format attendu pour les données envoyées par le client.

@QueryParam, @PathParam, @HeaderParam, etc. :

Permettent de lire les paramètres des requêtes.

Conteneurs compatibles JAX-RS :

Les services RESTful basés sur JAX-RS peuvent être déployés dans des conteneurs compatibles comme **Tomcat**, **WildFly**, ou **Spring Boot**.

Exemple de Classe avec annotations JAX-RS :

java

Copier le code

```
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import java.util.ArrayList;
import java.util.List;

@Path("/users") // Chemin de base pour cette ressource
@Produces(MediaType.APPLICATION_JSON) // Les réponses seront au format JSON
@Consumes(MediaType.APPLICATION_JSON) // Les données entrantes seront au format JSON
public class UserResource {

    private static List<User> users = new ArrayList<>();

    // Méthode pour récupérer tous les utilisateurs
    @GET
    public List<User> getAllUsers() {
        return users; // Retourne la liste d'utilisateurs
    }

    // Méthode pour récupérer un utilisateur par ID
    @GET
    @Path("/{id}") // Paramètre dynamique dans le chemin
    public User getUserById(@PathParam("id") int id) {
        return users.stream()
            .filter(user -> user.getId() == id)
            .findFirst()
            .orElseThrow(() -> new WebApplicationException("User not found", 404));
    }
}
```

```
// Méthode pour ajouter un nouvel utilisateur
@POST
public User addUser(User user) {
    user.setId(users.size() + 1); // Générer un ID simple
    users.add(user);
    return user; // Retourner l'utilisateur ajouté
}

// Méthode pour mettre à jour un utilisateur existant
@PUT
@Path("/{id}")
public User updateUser(@PathParam("id") int id, User updatedUser) {
    User user = users.stream()
        .filter(u -> u.getId() == id)
        .findFirst()
        .orElseThrow(() -> new WebApplicationException("User
not found", 404));
    user.setName(updatedUser.getName());
    user.setEmail(updatedUser.getEmail());
    return user;
}

// Méthode pour supprimer un utilisateur
@DELETE
@Path("/{id}")
public void deleteUser(@PathParam("id") int id) {
    users.removeIf(user -> user.getId() == id);
}
}
```

Services Web étendus VS REST

```
<?xml version="1.0" encoding="UTF-8"?>
<S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
<SOAP-ENV:Header/>
<S:Body> <ns2:hello xmlns:ns2=" http://uiz.ac.ma/ensag/gi/livres/">
<name>Miage NTDP</name>
</ns2:hello>
</S:Body>
</S:Envelope>
```



Client

SOAP



Serveur



Client

<http://uiz.ac.ma/ensag/gi/livres/1>

REST



Serveur

WADL

- ➔ **Web Application Definition Language** est un langage de description des services REST au format XML. Il est une spécification de W3C initié par SUN (www.w.org/Submission/wadl)
- ➔ Il décrit les éléments à partir de leur type (Ressources, Verbes, Paramètre, type de requête, Réponse)
- ➔ Il fournit les informations descriptives d'un service permettant de construire des applications clientes exploitant les services REST.

WADL

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
▼<application xmlns="http://wadl.dev.java.net/2009/02">
  <doc xmlns:jersey="http://jersey.java.net/" jersey:generatedBy="Jersey: 2.0 2013-05-03 14:50:15"/>
  <grammars/>
  ▼<resources base="http://localhost:8080/Bibliotheque/webresources/">
    ▼<resource path="category">
      ▼<method id="test" name="GET">
        ▼<response>
          <representation mediaType="application/xml"/>
          <representation mediaType="application/json"/>
        </response>
      </method>
    ▼<method id="apply" name="OPTIONS">
      ▼<request>
        <representation mediaType="*/*"/>
      </request>
      ▼<response>
        <representation mediaType="application/vnd.sun.wadl+xml"/>
      </response>
    </method>
  ▼<method id="apply" name="OPTIONS">
    ▼<request>
      <representation mediaType="*/*"/>
    </request>
    ▼<response>
      <representation mediaType="text/plain"/>
    </response>
  </resources>
</application>
```



```
▼<resource path="application.wadl">
  ▼<method id="getWadl" name="GET">
    ▼<response>
      <representation mediaType="application/vnd.sun.wadl+xml"/>
      <representation mediaType="application/xml"/>
    </response>
  </method>
▼<method id="apply" name="OPTIONS">
  ▼<request>
    <representation mediaType="*/*"/>
  </request>
  ▼<response>
    <representation mediaType="text/plain"/>
  </response>
</method>
▼<method id="apply" name="OPTIONS">
  ▼<request>
    <representation mediaType="*/*"/>
  </request>
  ▼<response>
    <representation mediaType="*/*"/>
  </response>
</method>
```

Développer des Web Services REST avec JAVA

Partie 2

Annotation JAX-RS

La spécification JAX-RS dispose d'un ensemble d'annotation permettant d'exposer une classe java dans un services web :

- ❑ @Path

- ❑ @GET, @POST, @PUT, @DELETE

- ❑ @Produces, @Consumes

- ❑ @PathParam

JAX-RS : @PATH

- ❑ L'annotation permet de rendre une classe accessible par une requête HTTP
- ❑ Elle définit la racine des ressources (Root Racine Ressources)
- ❑ La valeur donnée correspond à l'uri relative de la ressource

```
@Path("category")
public class CategoryFacade
{
.....
}
```

Adresse du
serveur

Ressource

<http://localhost:8080/Bibliotheque/webresources/category>

Port

Contexte de
l'application

JAX-RS : @PATH

- ❑ L'annotation peut être utilisée pour annoter des méthodes d'une classe
- ❑ L'URI résultante est la concaténation entre la valeur de @path de la classe et celle de la méthode

```
@Path("category")
public class CategoryFacade {
    @GET
    @Produces({MediaType.APPLICATION_XML, MediaType.APPLICATION_JSON})
    @Path("test")
    public String hello()
    {
        return "Hello World!";
    }
    ..
}
```

<http://localhost:8080/Bibliotheque/webresources/category/test>

JAX-RS : @PATH

- ❑ La valeur définie dans l'annotation **@Path** n'est forcément une constante, elle peut être variable.
- ❑ Possibilité de définir des expressions plus complexes, appelées Template Parameters
- ❑ Les contenus complexes sont délimités par « {} »
- ❑ Possibilité de mixer dans la valeur **@Path** des expressions

```
@GET
@Consumes ({MediaType.APPLICATION_JSON,
MediaType.APPLICATION_XML}) @Produces
({MediaType.APPLICATION_JSON,
MediaType.APPLICATION_XML})
@Path (" {nom} ")
public String hello (@PathParam("nom") String nom) {
    return "Hello " + nom;
}
```

@GET, @POST, @PUT, @DELETE

- Permettent de mapper une méthode à un type de requête HTTP
- Ne sont utilisables que sur des méthodes
- Le nom de la méthode n'a pas d'importance, JAX détermine la méthode à exécuter en fonction de la requête

<http://localhost:8080/Bibliotheque/webresources/category/test>

```
@Path("category")
public class CategoryFacade {
    @GET
    @Produces({MediaType.APPLICATION_XML,
    MediaType.APPLICATION_JSON})
    @Path("test")
    public String hello()
    {
        return "Hello World!";
    }
    ..
}
```

```
@GET
@Consumes ({MediaType.APPLICATION_JSON,
    MediaType.APPLICATION_XML}) @Produces
    ({MediaType.APPLICATION_JSON,
    MediaType.APPLICATION_XML}) @Path("{nom}")
    public String hello (@PathParam("nom")
    String nom){
        return "Hello " + nom;
    }
```

<http://localhost:8080/Bibliotheque/webresources/category/Ensag>

@GET, @POST, @PUT, @DELETE

- Les opérations CRUD sur les ressources sont réalisées au travers des méthodes de la requête HTTP



GET, POST
PUT, DELETE



/books

GET : Liste des livres

POST : Créer un **nouveau** livre

/books/{id}

GET : Livre identifié par l'id

PUT: Mis à jour du livre identifié par id

DELETE : Supprimer le livre identifié par id

@GET, @POST, @PUT, @DELETE

```
@Path("livre")
```

```
public class LivreFacadeREST extends AbstractFacade<Livre> {
```

```
@POST @Override
```

```
@Consumes({"application/xml", "application/json"})
```

```
public void create(Livre entity) {
```

```
    super.create(entity); }
```

```
@PUT @Override
```

```
@Consumes({"application/xml", "application/json"})
```

```
public void edit(Livre entity) {
```

```
    super.edit(entity); }
```

```
@DELETE @Path("{id}")
```

```
public void remove(@PathParam("id") Long id) {
```

```
    super.remove(super.find(id)); }
```

```
@GET @Path("{id}")
```

```
@Produces({"application/xml", "application/json"})
```

```
public Livre find(@PathParam("id") Long id) {
```

```
    return super.find(id); }
```

```
@GET @Override
```

```
@Produces({"application/xml", "application/json"})
```

```
public List<Livre> findAll() {
```

```
    return super.findAll(); }
```

```
@GET @Path("{from}/{to}") @Produces({"application/xml", "application/json"})
```

```
public List<Livre> findRange(@PathParam("from") Integer from, @PathParam("to") Integer to) {
```

```
    return super.findRange(new int[]{from, to}); }
```

Paramètres des requêtes

- JAX-RS fournit des mécanismes pour extraire des paramètres dans la requête
- Utilisés sur les paramètres des méthodes des ressources pour réaliser des injections de contenu
- Différentes annotations :
 - @PathParam : valeurs dans templates parameters
 - @QueryParam : valeurs des paramètres de la requête
 - @FormParam : Valeurs des paramètres de formulaire
 - @HeaderParam: Valeurs dans l'en tête de la requête
 - @CookieParam : Valeurs des cookies
 - @Context : Informations liés au contexte de la ressource

Exemple de web service RESTful

Consulter un compte

Consulter la liste des comptes

Effectuer un versement

Effectuer un retrait

Effectuer un virement

Supprimer un compte

Exemple de web service JAX-RS

```
package ma.bp.banqueWS.service;
import java.util.*; import javax.ws.rs.*;
import javax.ws.rs.core.MediaType;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import ma.bp.banqueWS.entities.Compte;
import ma.bp.banqueWS.metier.IBanqueMetier;
@Component
@Path("/banque")
public class BanqueRestService { public class BanqueRestService {
    @Autowired
    IBanqueMetier metier;
    @POST
    @Path("/comptes")
    public void addCompte(@FormParam(value="solde")double solde){
        metier.addCompte(new Compte(null, solde,new Date()));
    }
}
```

Exemple de web service JAX R

GET

@Path("/comptes/{code}")

@Produces(value={MediaType.APPLICATION_JSON,MediaType.APPLICATION_XML})

public Compte getCompte(@PathParam(value="code")Long code){

return metier.getCompte(code);

}

@GET

@Path("/comptes")

@Produces(value={MediaType.APPLICATION_JSON,MediaType.APPLICATION_XML})

public List<Compte> getComptes(){ public List<Compte> getComptes(){

return metier.listComptes();

}

@PUT

@Path("/comptes/verser")

**public void verser(@FormParam("code") Long code,@FormParam("montant") double
montant){**

metier.verser(code, montant);

}

Exemple de web service JAX R

```
@PUT
@Path("/comptes/retirer")
public void retirer(@FormParam("code") Long code,@FormParam("montant")
double montant){
    metier.retirer(code, montant);
}

@PUT
@Path("/comptes/virement")
@Produces(value={MediaType.APPLICATION_JSON,MediaType.APPLICATION_XML})
public void virement(@FormParam("cpte1") Long cpte1, public void
virement(@FormParam("cpte1") Long cpte1,
@FormParam("cpte2") Long cpte2,
@FormParam("montant") double montant){
    metier.virement(cpte1, cpte2, montant);
}

@DELETE
@Path("/comptes/{code}")
public void supprimer(@PathParam("code")Long code){
    metier.deleteCompte(code);
}}
```